AI-DRIVEN SOFTWARE ENGINEERING · FALL 2026 · KBTU SITE

Practice #07 — Testing AI-Generated Code

1 point · branch and folder `week-07` · AI use **Level D** · due **Friday 6 November, 23:59** — Practice #08 runs this suite, so finish this one first.

Continue your Week 06 project. Pull the booking decision out into `canBook`, let an assistant write the tests, and judge them: expected values from the criteria, a real run, three planted defects.

Fixed: the runner is **Vitest**; the interface, T01–T14, the two deck prompts, M1–M3. **Your choice:** the assistant (Bolt's own chat is easiest). **No Week 06 project?** Use `week-07/starter/` and declare it — it costs nothing.

The function everyone tests

```
export function canBook({ start, end, now, blocked, bookings }): boolean
// minutes after midnight (600 = 10:00, 1440 = 24:00); true = Available, false = Unavailable
// in app/src/booking.ts (or .js) — and the page must call it
```

The plan — default fixture: now 540, not blocked, bookings [{ start: 600, end: 660 }]

ID	Input	Exp.	AC	ID	Input	Exp.	AC
T01	660–720 touching after	true	AC4	T08	570–600 touching before	true	AC4
T02	540–570 starts now	false	AC1	T09	630–690 partial overlap	false	AC4
T03	720–720 zero duration	false	AC1	T10	570–690 contains booking	false	AC4
T04	750–720 reversed	false	AC1	T11	615–645 inside booking	false	AC4
T05	1380–1441 past day end	false	AC1	T12	600–660 identical	false	AC4
T06	720–840 (120 min)	true	AC2	T13	660–720, blocked = true	false	AC3
T07	720–841 (121 min)	false	AC2	T14	630–690, bookings = []	true	AC4

Plus: an **AC5** test (an Available request, bookings compared with a copy taken before) and **X1, X2** — two cases of your own. At least 17 tests. **The Expected column comes from the ACs, never from the code.**

The steps

1. **Setup (in class).** Branch `week-07`; bring your project in first: `mkdir week-07 && git checkout week-06 -- week-06/prototype && git mv week-06/prototype week-07/app`; then unzip. Commit the baseline.
2. **Task 1 — extract (in class).** Move the decision into `canBook` without changing behaviour; the page imports it. `npm install -D vitest`.
3. **Task 2 — plan (in class).** A reason for every case, in your words, before any assistant. Add X1, X2.
4. **Task 3 — generate (in class).** The deck's prompt, `[our test framework]` → Vitest, plus your table. Save the file unedited to `suites/first/`.
5. **Task 4 — review before running.** No-exception tests, expected = current output, only valid requests, a mocked function. Two decisions or more, each with an AC.
6. **Task 5 — run, diagnose, improve.** `npx vitest run` — paste the real output. Test repair or implementation fix? Then the deck's improvement prompt.
7. **Task 6 — defects.** M1 `>= 120` · M2 touching as overlap · M3 no blocked check. One at a time, tests unchanged, record, restore.
8. **Task 7 — UI by hand.** Five checks, `evidence/ui-1 … ui-5`, ui-4 at phone width (portrait).

Rules the checker reads

- One test per case, the ID in its title: `test('T05 AC1: …')`. No `test.each`, `.skip`, `.only`, `vi.mock`.
- Exact decisions: `.toBe(true)` / `.toBe(false)` — never `toBeTruthy`. `now` is a number: no `Date.now()`.
- Your pasted final run must report as many tests as your committed files hold. A build message or "tests pass" is not a run.

VERDICTS AND COMMANDS

```
PASS   the check found what it was looking for
FAIL   the content is there but wrong, or not enough of it
ERROR  it could not check: a file, a folder or a section is missing

cd week-07 && python tests/check_suite.py          (33 checks)
            python tests/validate_submission.py
```

Submit and grading

PR `week-07 → main`, left open; the PR link in the Teams assignment **Practice #07**. Not an e-mail.

Criterion (0.25 each)	Complete when …
Test plan and suite	canBook extracted and called by the page; a reason for every case; T01–T14, AC5, X1, X2 exact and on the plan's inputs; the real function, no mocks, no clock.
Runs and mutants	Real first and final runs; every failure diagnosed; M1–M3 tried, each detected or explained; five UI checks with screenshots.
Review decisions	First suite kept as generated; two decisions with an AC; both prompts as written; no correct expectation bent to the code.
Evidence and honesty	§9, §10 your own; AI_USAGE.md; submission.yml validating and matching the re-run; open PR, 3+ commits.

A FAIL you report and explain costs you nothing. One you hide costs the whole criterion.